

Prolog Arcade

Trabalho de Programação em Lógica

Realizado pelos alunos:

Bruno Barros(up202305457),
Francisco Moreira (up202306691),

Índice

1. Introdução (Pág. 3)
2. Requisitos/Dependências do Sistema (Pág. 3)
3. Como configurar (Pág. 3)
4. Implementação (Págs. 3 a 5)
5. Resultados (Págs. 6 a 10)
6. Escalabilidade (Págs. 10 e 11)
7. Conclusão (Pág. 11 e 12)
8. Referências (Pág. 12)

1. Introdução

O seguinte projeto foi desenvolvido em contexto académico, de forma a consolidar competências de programação lógica. Para tal efeito, foi criado um sistema integrado de jogos e puzzles - o Prolog Arcade - com interface web e comunicação com APIs desenvolvidas em SWI-Prolog.

O sistema divide as responsabilidades por vários servidores independentes (microserviços) responsáveis pela lógica de jogos como Sudoku, Minesweeper, Star Battle e 2048, interligando a inteligência dos motores Prolog com uma interface frontend em HTML/JS.

2. Requisitos/Dependências do Sistema

- OS: Windows/ Linux/ MacOS.
- SWI-Prolog instalado no PATH do sistema (para funcionar no terminal).

3. Como configurar

- Descarregue a pasta.
- Aceda ao diretório
- Clique no batch Start Servers (inicializar os servidores nas respetivas portas)
- Abra o site games para conseguir jogar os jogos disponíveis.
- Alternativamente, aceda ao diretório no terminal e execute os servidores individualmente:
Por exemplo: Sudoku: `swipl -s sudoku.pl` (porta 8080 no localhost)

4. Implementação

Resumo dos Jogos (Prolog Arcade)

O **Prolog Arcade** reúne quatro quebra-cabeças clássicos de raciocínio lógico, reinventados e integrados para demonstrar o poder computacional do SWI-Prolog como motor de *back-end*:

- **Sudoku**: O famoso *puzzle* de preenchimento de grelhas 9x9, onde o objetivo é deduzir a posição dos números garantindo que nenhum se repete nas linhas, colunas ou blocos.
- **Minesweeper** (Campo Minado): O objetivo é deduzir a localização exata das minas escondidas, utilizando como pistas os números revelados nas casas, que indicam a quantidade de minas adjacentes.
- **Star Battle**: Um desafio lógico complexo onde o jogador tem de colocar estrelas num tabuleiro dividido em áreas irregulares. As estrelas estão submetidas a restrições estritas: não se podem tocar (nem sequer na diagonal) e cada linha, coluna e região tem de ter o número exato de estrelas definido.
- **2048**: Um jogo dinâmico de fusão e deslize de blocos num tabuleiro 4x4, onde peças iguais se somam numa peça de valor dobrado. No que toca ao limite de jogadas do 2048, o jogo não possui um contador máximo de turnos predefinido; em vez disso, as jogadas são puramente limitadas pelo espaço na matriz. O fim absoluto das jogadas (o estado de *Game Over*) é ativado estritamente quando o tabuleiro atinge a ocupação total (sem espaços vazios) e o motor Prolog avalia que esgotaste a possibilidade de fazer qualquer fusão adjacente (vertical ou horizontal).

Explicação do código no geral:

Cabeçalho:

Para este sistema foram usadas variadas bibliotecas *built-in*, tais como:

- **library(http/thread_httpd)**: inicia os servidores HTTP multithread;
- **library(http/http_dispatch)**: define *handlers* para as rotas e *endpoints*;
- **library(http/http_cors)**: permite a partilha de recursos (*CORS*) entre origens, vital para que o *frontend* estático comunique com os diversos portos dos jogos;
- **library(http/http_json)**: suporta a leitura e escrita em objetos JSON, para o intercâmbio rápido das matrizes dos tabuleiros com o *frontend*;
- **library(http/http_files)**: serve ficheiros estáticos simples (HTML, CSS, JS);
- **library(clpfd)**: biblioteca de programação lógica com restrições (*Constraint Logic Programming over Finite Domains*), que funciona como o motor dedutivo principal para resolver os puzzles lógicos;
- **library(lists)**: essencial para manipulação, agregação e verificação de matrizes e listas dentro dos tabuleiros.

Devido à arquitetura descentralizada (microserviços), o sistema foi dividido em módulos independentes através do predicado `module/1`. Cada puzzle possui o seu próprio ficheiro e servidor independente (`sudoku.pl`, `minesweeper.pl`, `star.pl`, `2048.pl`).

O predicado `initialization/1` (ou através da chamada direta pelo nosso *script* de *batch* `start_servers.bat`) é utilizado para inicializar rapidamente cada servidor no seu porto designado (ex: 8080 para Sudoku, 8082 para Minesweeper), colocando as APIs imediatamente à escuta de pedidos via `http_server/2`.

Foi utilizado o predicado `http_handler/3` com a função de processar pedidos de *endpoints* individuais. No ficheiro central `serve.pl`, este processa os ficheiros estáticos entregando a *interface* visual (`games.html`). Nos motores lógicos dos jogos (como as rotas para `/solve` ou `/hint`), os *handlers* processam o corpo do pedido em JSON, executam o motor matemático do Prolog, e devolvem a solução ou validação nesse mesmo formato.

Layout:

Ao contrário de gerar o HTML de forma algorítmica no Prolog (via DCGs), o projeto focou-se numa separação moderna de responsabilidades. O *layout* foi construído em ficheiros autónomos em HTML5 e CSS3 (com propriedades visuais, tipografia interativa e animações) que operam no navegador de forma assíncrona (*Fetch API*), consumindo o Prolog como uma base computacional externa pura.

Server:

Para gerar as sessões HTTP de cada módulo, invocamos `http_server(http_dispatch, [port(Port)])`. Através do `start_servers.bat`, cada motor é isolado no seu próprio porto, permitindo escalabilidade e

concorrência simultânea e independente (um cálculo massivo num jogo não interfere com os restantes).

Motores de Puzzles (Solvers com CLP(FD)):

A base deste projeto não são bases de dados armazenadas, mas sim a dedução através de *Constraint Logic Programming*.

- **Sudoku (sudoku.pl):** O motor recebe o tabuleiro e mapeia as casas vazias para variáveis em domínios de 1 a 9 (ins 1..9). Aplica restrições severas garantindo que as variáveis são unicamente distintas em linhas, colunas e blocos 3x3 através de matrizes com `all_distinct/1`. De seguida, invoca o enumerador `label/1` que encontra, instantaneamente, a solução do *puzzle*.
- **Minesweeper (minesweeper.pl):** Trata cada casa oculta como uma variável binária (0 ou 1). Através das regras matemáticas dos vizinhos revelados que contêm números, impõe restrições de soma a essas variáveis e enumera soluções em `clpfd`. Consequentemente, consegue deduzir infalivelmente se uma casa é segura (se for 0 em todos os cenários possíveis) ou se contém uma mina (1).

Outros Jogos (Star Battle e 2048):

Para mecânicas dinâmicas de deslize como no **2048** (2048.pl), o Prolog executa matrizes numéricas, recursão de regras como remoção de zeros, agrupamento consecutivo (`merge_row`) e preenchimento vetorial (`slide_left_board`). As listas são invertidas em múltiplos ângulos de forma lógica para simular o movimento espacial.

Interação Dinâmica (Hints e Validações):

Para não retirar valor à experiência de jogo fornecendo apenas resoluções completas, as lógicas foram moduladas. O sistema providencia *endpoints* secundários para dicas instantâneas, onde o Prolog fornece unicamente a posição correta de uma peça (recorrendo ao mesmo *solver*, mas apenas partilhando um facto pontual). O sistema permite também uma validação estática das jogadas atuais, conferindo em tempo real se a tentativa do utilizador contrapõe alguma regra matemática.

5. Resultados:

5. Resultados

Interface desenvolvida em HTML e JavaScript servindo de apoio a Prolog usada no projeto

Página Principal (Arcade Hub)

Apresenta o ambiente de jogos com a opção para seleccionar e navegar pelas vertentes implementadas, exibindo um design focado na facilidade de navegação

e atração visual.



Figura 1: Página inicial (Arcade Hub), desenvolvida com design moderno, servindo como ponto de acesso centralizado aos vários microsserviços do sistema.

Sudoku e o Motor Computacional em Ação

Os tabuleiros são processados graficamente no browser e as interações do utilizador enviam pedidos instantâneos ao servidor Prolog. A mecânica permite não só jogar, mas invocar o motor *clpfd* para obter soluções ou dicas em tempo real.

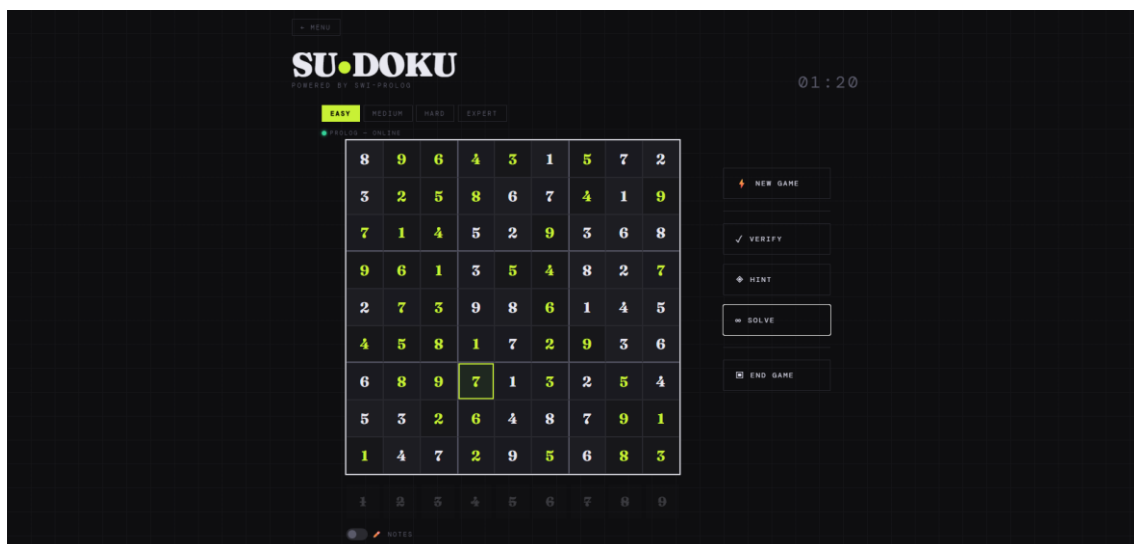
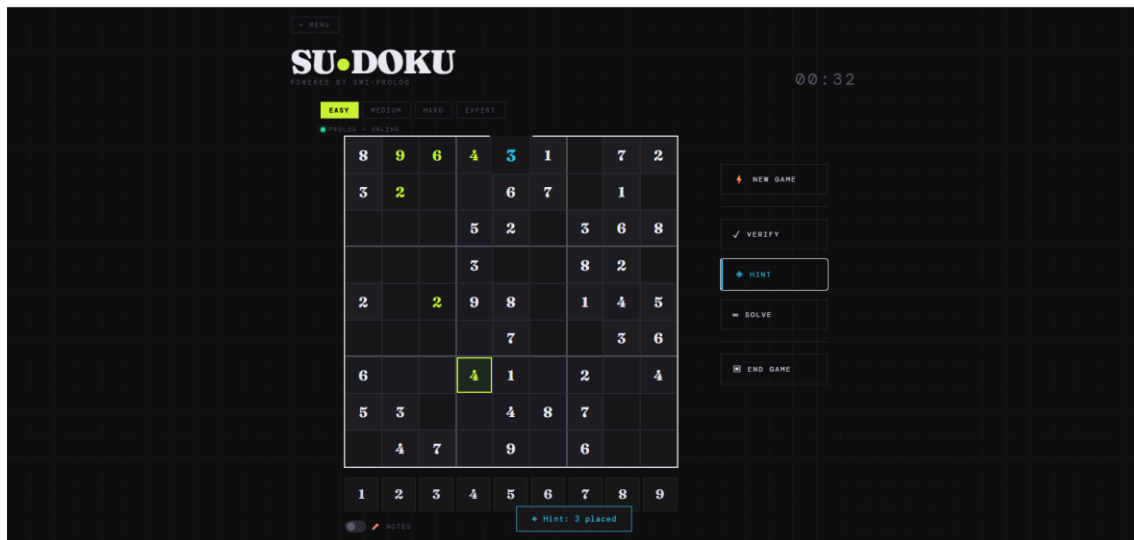


Figura 2 e 3: Interface do Sudoku. Demonstração das células deduzidas instantaneamente após o utilizador invocar o motor computacional clpfd através da opção de ajuda.

Minesweeper e a Lógica Dedutiva

O campo minado ilustra a aplicação de restrições de vizinhança. O Prolog processa e resolve logicamente a localização de casas livres e armadilhadas através do processamento JSON sem necessidade de adivinhação.

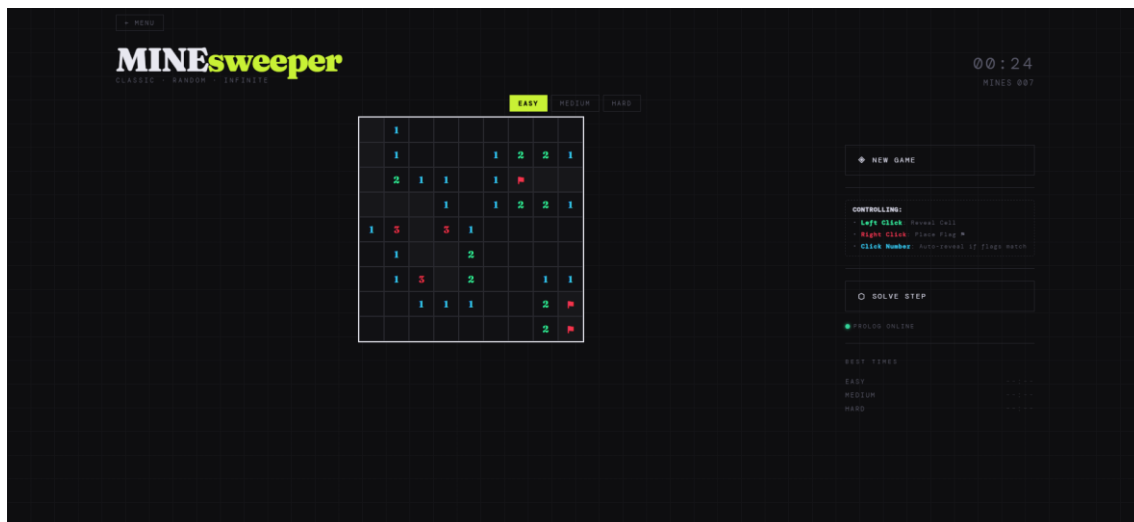


Figura 4: Tabuleiro de Campo Minado, evidenciando as bandeiras colocadas e as zonas seguras descobertas graças às restrições avaliadas pelo servidor Prolog.

A Matriz Dinâmica do Jogo 2048

Nos jogos de deslize progressivo, as respostas assíncronas do backend gerem fusões consecutivas numa matriz que progressivamente atinge o seu limite físico de espaço.

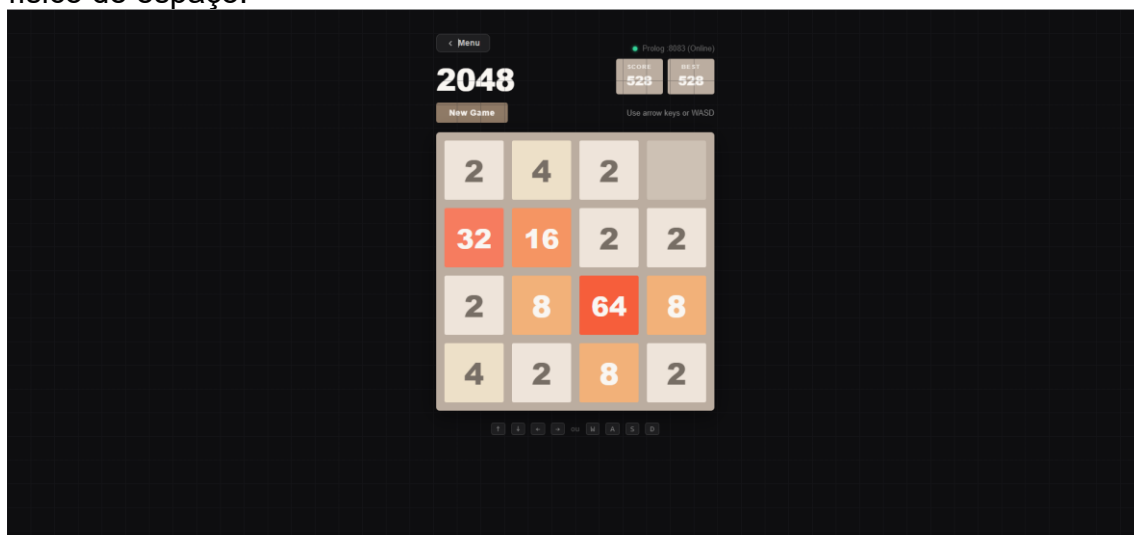


Figura 5: Partida avançada de 2048. O motor Prolog calcula a nova grelha em milissegundos após cada deslize, perto de atingir o estado de Game Over por limite espacial.

Star Battle e as Restrições Visuais

A validação de jogadas num ambiente não retangular demonstra como a comunicação entre front-end e o backend isolado avalia condições complexas num piscar de olhos.

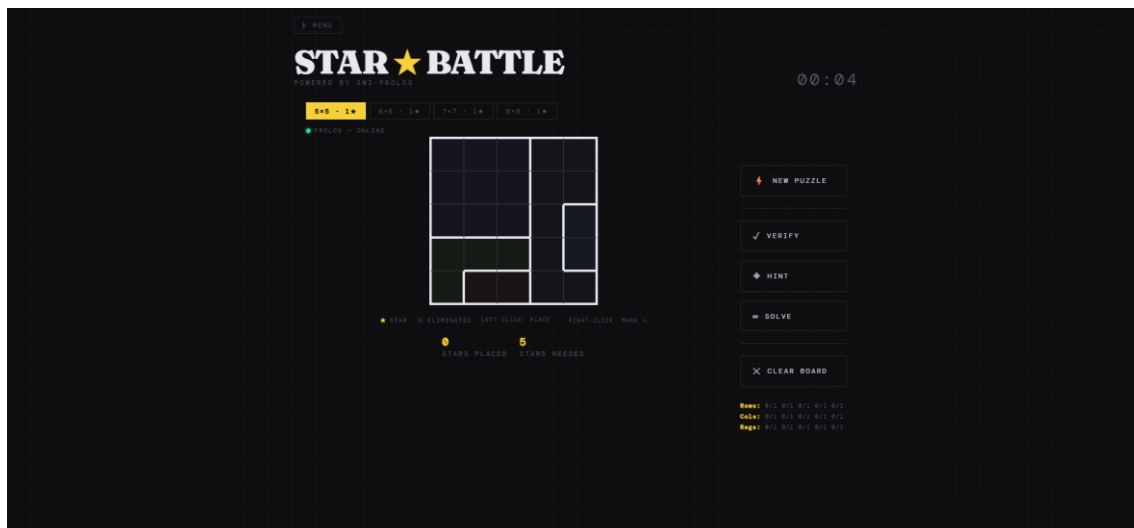


Figura 5: Interface do Star Battle, onde a validade de colocação isolada e correta das estrelas é assegurada pelas restrições lógicas do back-end.

6. Escalabilidade

Embora o sistema se encontre numa fase inicial, a sua arquitetura baseia-se em conceitos que permitem um crescimento sustentável, focando-se no processamento intensivo (CPU) em vez do tradicional armazenamento de dados.

Gestão da Complexidade e Volume Computacional

Ao contrário de sistemas geridos por grandes bases de dados, o principal desafio deste projeto não é o volume de dados alojados, mas sim a carga computacional gerada pela resolução de problemas NP-Completo (como o Sudoku e Star Battle) em simultâneo. À medida que o sistema adotar milhares de utilizadores a enviar tabuleiros, manter o processamento unicamente nas *threads* base pode causar atrasos e bloqueios. Uma mitigação futura passa por integrar filas de espera (*Message Queues*, como RabbitMQ ou Redis) para gerir os pedidos de cálculos muito pesados ordenadamente, sem bloquear o *front-end*.

Otimização do Motor Prolog

O SWI-Prolog aloca *threads* web eficientemente, contudo o motor de restrições (clpfd) consome ciclos de CPU elevados ao enumerar as combinações lógicas. Para otimizar a resposta, pode-se evoluir adotando *Thread Pools* rígidas, que limitam o número máximo de resoluções concorrentes para não esgotar a máquina. Adicionalmente, pode-se compilar os ficheiros .pl em binários otimizados (extensão .qlf - *Quick Load Files*), eliminando a latência de compilação em *runtime* nos reinícios do servidor.

Caching e Pré-computação

Para reduzir a latência drástica e a carga do processador central, é possível recorrer a uma *cache* de resultados. Quando diferentes utilizadores pedem ajuda para um estado de tabuleiro específico (idêntico ao de outro jogador no passado), a matriz com a solução ou a dica gerada pode ser guardada temporariamente. Desta forma, o pedido não invoca o algoritmo dedutivo do Prolog na totalidade, retornando o resultado em memória instantaneamente.

Arquitetura Distribuída

Separar responsabilidades evita gargalos num único processo, e o projeto já adotou esse caminho separando os portos dos jogos (Sudoku no 8080, Campo Minado no 8082, etc.). Num futuro em produção, cada um destes módulos funcionará num *container* isolado (microsserviços puros). A entrega de HTML/Frontend poderá ser totalmente dissociada para um servidor estático altamente performante (como NGINX), enquanto as APIs em Prolog lidam estritamente com os cálculos JSON via REST.

Escalabilidade Horizontal

Devido à natureza *stateless* (sem necessidade de guardar dados persistentes de estado na memória) dos nossos motores *clpfd*, atingir a escalabilidade horizontal é trivial. Podemos ter múltiplas réplicas do *sudoku.pl* ou *minesweeper.pl* a correr paralelamente atrás de um balanceador de carga. Se o tráfego aumentar subitamente, as instâncias são multiplicadas sem conflitos de sessão.

Proteção contra Sobrecarga (Rate Limiting)

Como os cálculos são densos, o sistema fica vulnerável a exaustão de CPU caso haja pedidos automatizados. Em vez de lidar com os limites da API, a nossa arquitetura deve implementar um *middleware* inteligente que deteta excessos, aplicando um bloqueio (*Rate Limit*) que limite os utilizadores a X pedidos de "Solve" por minuto, respondendo com o código *HTTP 429 Too Many Requests* antes que os pedidos atinjam os motores lógicos do Prolog.

Testes de Carga e Benchmarking

É necessário também validar empiricamente a escalabilidade do sistema antes de entrar em produção, usando cenários de *stress*, isto é, simular centenas ou milhares de utilizadores concorrentes a pedir resoluções simultâneas de tabuleiros complexos. Isto serve para determinar se a adição de mais réplicas reduz linearmente a latência, ou se surgem novos gargalos de I/O, ou mesmo utilizar testes de resiliência que desligam instâncias a quente (*chaos testing*) para observar a recuperação automática dos processos Prolog sob *stress*.

7. Conclusão

Em conclusão, a criação de um sistema de Arcade em Prolog permitiu-nos aprimorar competências de programação lógica e estimular criatividade, tal como aprender técnicas como a integração de APIs externas e a gestão de factos Prolog. Constatou-se que o sistema apresenta elevada precisão nas recomendações, mas também algumas limitações ao nível da escalabilidade. No futuro, este projeto poderá evoluir não só com as estratégias descritas no capítulo de escalabilidade, mas também através de integrações com aplicações móveis ou outras plataformas, caminhando assim para um produto real.

8. Referências

kaggle.com. (n.d.). *IMDb movies extensive dataset*. [online] Available at: <https://www.kaggle.com/stefanoleone992/imdb-extensive-dataset>.

The Movie Database (2018). *The Movie Database*. [online] Themoviedb.org. Available at: <https://www.themoviedb.org/>.

Wilk, J. (2023). *How to Build a Movie Recommendation System Based on Collaborative Filtering*. [online] freeCodeCamp.org. Available at: <https://www.freecodecamp.org/news/how-to-build-a-movie-recommendation-system-based-on-collaborative-filtering/>.